

análisis de SonarQube. Para BMAD se reporta únicamente la ejecución con planificación completa: *product brief*, PRD, arquitectura, épicas e historias, construcción y corrección de alcance. Se utilizó BMAD Method 6.12.0 con los módulos *core* y *bmm* y la integración Gemini; la semilla y el alcance permanecieron iguales a los de las otras cuatro herramientas.

La Tabla 9.13 precisa las versiones utilizadas en estas ejecuciones, que difieren de algunas versiones de referencia de la Tabla 9.4. En particular, la ejecución de Tessl utilizó la CLI vigente como gestor de contexto, con plugins públicos para FastAPI y React y su integración MCP con Gemini; no instaló el *tile* SDD 2.0.1 presentado como referencia teórica en el capítulo. Esta diferencia metodológica debe conservarse al interpretar sus resultados.

Tabla 9.13: Versiones efectivas de las herramientas en Tetris Etapa 1

Componente	Versión observada
OpenSpec	1.13.2
GitHub Spec Kit	1.0.12
Tessl CLI	0.108.0; plugins de contexto públicos y MCP
BMAD Method	6.12.0
Kiro CLI	2.25.0 (modo Auto)
Gemini CLI	0.60.0 (OpenSpec, Spec Kit, Tessl y BMAD)
SonarQube	Community Build 26.9.0.129388

Dimensión A: Esfuerzo Computacional y Costos

La Tabla 9.14 sigue las categorías empleadas para la calculadora. Los cuatro flujos basados en Gemini CLI utilizaron Vertex AI; Kiro CLI se ejecutó en modo *Auto*. Los valores precedidos por $\geq$ son el tráfico mínimo recuperable de los registros por ejecución, no el consumo facturado ni un conteo completo de la etapa: falta el registro de tokens de una ejecución de OpenSpec, una de Spec Kit y tres de Tessl. Se distingue la entrada total de la porción cacheada para no cobrar dos veces los mismos tokens. La estimación económica aplica la fórmula y las tarifas indicadas antes en este capítulo únicamente al tráfico observado; no equivale a una factura de GCP, que puede estar cubierta por créditos del proyecto. El protocolo ACP conservado de Kiro no registra tokens ni créditos consumidos de forma verificable, por lo que tampoco permite calcular su costo.

Tabla 9.14: Tráfico observable y costo teórico del modelo – Inicialización (Tetris Web)

Herramienta	Input tokens	Input cacheado	Output tokens	Créditos	Costo observable (USD)
OpenSpec	$\geq 5.816.534$	$\geq 4.217.424$	≥ 39.707	N/A	$\geq 3,3886$
Spec Kit	$\geq 14.171.471$	$\geq 10.956.018$	≥ 90.198	N/A	$\geq 7,2784$
Tessl	$\geq 5.952.918$	$\geq 3.896.072$	≥ 41.324	N/A	$\geq 4,0416$
BMAD	17.653.626	12.533.513	237.579	N/A	11,6984
Kiro	N/D	N/D	N/D	N/D	N/D

Nota: N/A significa que la plataforma no usa esa unidad; N/D indica que la ejecución no dejó un dato verificable. En OpenSpec, Spec Kit y Tessl faltan 1/6, 1/6 y 3/8 registros de tokens, respectivamente. Los mínimos se refieren a sesiones registradas y no a la factura total.

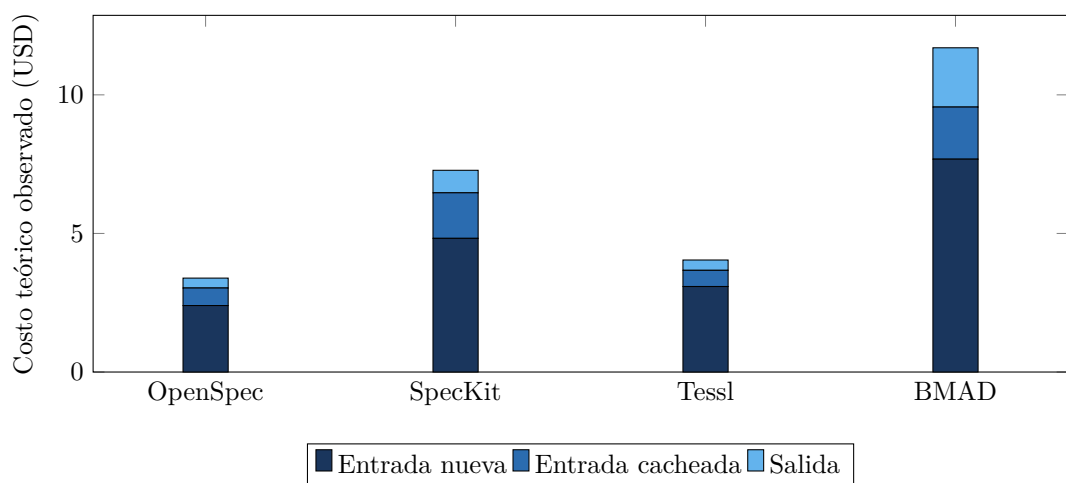


Figura 9.7: Desglose del costo teórico del tráfico de tokens recuperable. Las barras incompletas de OpenSpec, Spec Kit y Tessl representan mínimos; Kiro carece de datos verificables.

Entre las sesiones medibles, BMAD registra la mayor entrada observada (17,65 millones de tokens) y el mayor costo teórico del tráfico (11,70 USD). Esta cifra suma entrada nueva, entrada cacheada y salida con las tarifas declaradas; no corresponde a una factura de GCP. Los importes de OpenSpec, Spec Kit y Tessl son mínimos por sus registros incompletos, de modo que la comparación económica no identifica un costo final ordenado para las cinco herramientas. Kiro permanece fuera de esta comparación monetaria por falta de créditos trazables, aunque su ejecución y sus artefactos son observables.

Tabla 9.15: Esfuerzo operativo registrado – Inicialización (Tetris Web)

Herramienta	Solicitudes registradas	Tiempo de agente (s)	Llamados a herramientas	Sesiones sin tokens
OpenSpec	6	3.065	≥ 85	1
Spec Kit	6	3.359	≥ 162	1
Tessl	8	4.901	≥ 86	3
BMAD	9	3.299,511	≥ 342	2
Kiro	1	546	57	N/A

Nota: Solicitud = invocación registrada del agente, no cada mensaje interno del modelo. El tiempo suma la duración de esas invocaciones, incluso intentos fallidos; excluye instalaciones, pruebas independientes y escaneo. Los llamados mínimos de Gemini solo cubren salidas JSON con estadística de herramientas; en Kiro se contaron eventos `tool_call` del protocolo ACP. Estas definiciones difieren de las interacciones por mensaje usadas para Calculadora, por lo cual sus valores absolutos no deben compararse directamente entre casos.

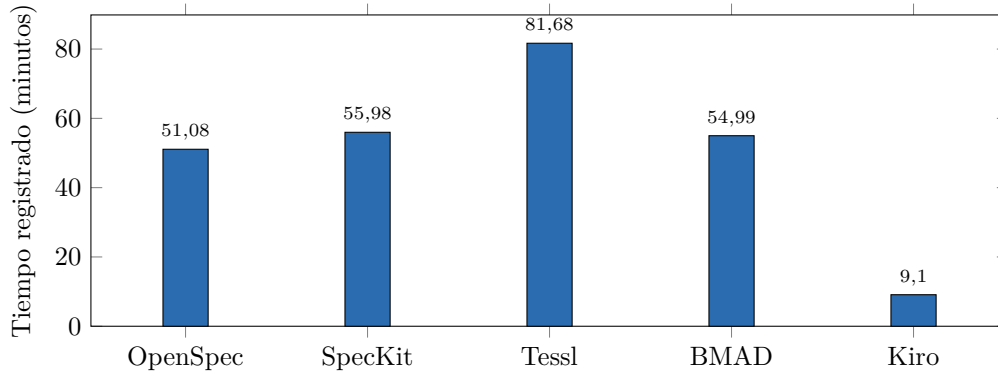


Figura 9.8: Duración acumulada de las invocaciones del agente en Tetris, sin tiempo de instalación ni validación externa.

Tessl exigió ocho solicitudes y 81,7 minutos de agente, el máximo del grupo; su ejecución incluyó respuestas de Vertex con error 429 o planes sin archivos, y después una corrección para restringir los controles a las flechas exigidas por la semilla. Kiro terminó su flujo nativo en una invocación de 9,1 minutos, aunque ese tiempo no representa el esfuerzo total de preparación del entorno. OpenSpec y Spec Kit requirieron seis invocaciones cada uno. En Spec Kit se corrigió una prueba de aceptación imposible para una pieza I vertical sin rotación; en BMAD se preservaron 30 requisitos funcionales y se eliminaron metas de capacidad y latencia añadidas por el agente sin estar en la semilla. Sus nueve invocaciones incluyen un arranque fallido por permisos y una reanudación vacía sin tokens, además de las fases de planificación y construcción. Por tanto, la duración refleja tanto la herramienta como la necesidad de corrección y las condiciones de servicio; no debe interpretarse como una medición aislada de velocidad de generación.

Dimensión B: Calidad del Código Generado (SonarQube)

Las cinco variantes superaron su *quality gate* en SonarQube Community Build 26.9.0.129388. La Tabla 9.16 usa `ncloc` para LOC, las categorías clásicas `bugs`, `vulnerabilities` y `code_smells`, y los ratings Security/Reliability/Maintainability obtenidos de la API. Un valor 1 equivale a A y 3 a C. El *quality gate* aprobado no implica ausencia de hallazgos en el código total: SonarQube puede evaluar condiciones de código nuevo distintas de las cifras globales aquí presentadas.

Tabla 9.16: Métricas estáticas SonarQube – Inicialización (Tetris Web)

Herramienta	LOC	Bugs	Vulns.	Smells	Deuda (m)	Comp. cog.	Dupl.	Rating (S/R/M)
OpenSpec	898	0	1	15	81	76	0 %	C/A/A
Spec Kit	748	0	2	7	35	78	0 %	C/A/A
Tessl	788	0	1	13	71	69	0 %	C/A/A
BMAD	669	0	1	13	84	97	0 %	C/A/A
Kiro	497	0	0	10	38	45	0 %	A/A/A

Para construir un radar comparable en estructura con la Figura 9.4, se usaron los mismos cinco ejes y se normalizaron por separado los resultados de Tetris. En cada eje donde los valores difieren, una magnitud menor recibe mejor puntuación mediante $s(x) = 2 + 8(x_{\text{máx}} - x)/(x_{\text{máx}} - x_{\text{mín}})$; si todos los valores son iguales, se asigna $s = 10$ a las cinco variantes. La escala es relativa a este caso de estudio y no constituye una calificación SonarQube ni debe compararse numéricamente con la escala normalizada para Calculadora.

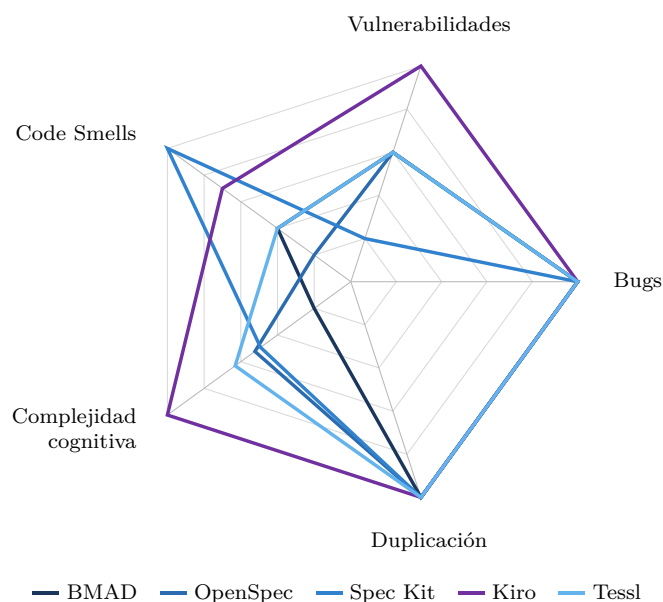


Figura 9.9: Radar de calidad estática normalizada – Inicialización Tetris. El exterior representa menos hallazgos o menor complejidad dentro de este caso. Bugs y duplicación valen 10 para todos porque sus valores observados fueron cero en las cinco variantes.

Tabla 9.17: Densidad de hallazgos y cobertura observada – Inicialización (Tetris Web)

Herramienta	Hallazgos clásicos	Issues/KLOC	Cobertura global	Pruebas backend exitosas
OpenSpec	16	17,82	60,2 %	13/13
Spec Kit	9	12,03	62,0 %	18/18
Tessl	14	17,77	60,4 %	9/9
BMAD	14	20,93	63,2 %	13/13
Kiro	10	20,12	72,6 %	39/39

Nota: Hallazgos clásicos = bugs + vulnerabilidades + *code smells*; Issues/KLOC = hallazgos clásicos / (ncloc/1000). La cobertura se informa como evidencia complementaria, sin utilizarla para puntuar esta etapa según el protocolo de las Etapas 1 y 2. La cantidad de pruebas varía entre variantes y no constituye por sí sola una tasa de aceptación funcional común.

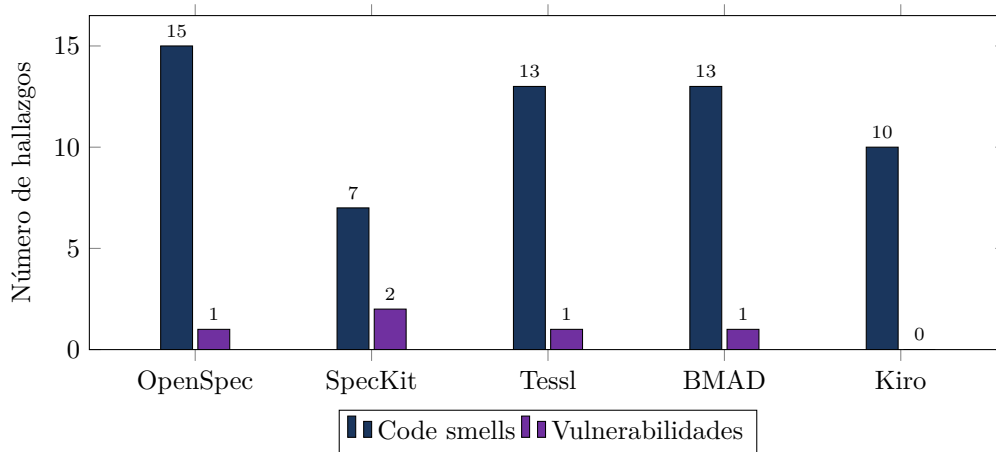


Figura 9.10: Hallazgos clásicos de SonarQube en Tetris. Ninguna variante registró bugs clásicos; los hallazgos se concentran en mantenibilidad y seguridad.

La Figura 9.9 muestra a Kiro en el extremo favorable para vulnerabilidades y complejidad cognitiva, mientras Spec Kit encabeza el eje de *code smells*. Kiro produjo el menor volumen de código medido por SonarQube (497 LOC), la menor complejidad cognitiva (45) y ninguna vulnerabilidad; es la única variante con rating A/A/A. Sin embargo, sus diez *code smells* repartidos en una base menor elevan su densidad a 20,12 hallazgos/KLOC, una de las más altas del conjunto. Por eso el radar de recuentos absolutos debe leerse junto con la densidad de la Tabla 9.17. Spec Kit tiene el menor número absoluto de hallazgos (nueve) y la menor deuda técnica (35 minutos). BMAD presenta 14 hallazgos clásicos, 20,93 por mil LOC, la mayor complejidad cognitiva (97) y la mayor deuda estimada (84 minutos), aunque su *quality gate* también fue aprobado. OpenSpec registra 15 *code smells*, el máximo del grupo. La duplicación medida fue 0% en las cinco variantes, por lo que no discrimina entre ellas en esta etapa. Las vulnerabilidades detectadas incluyen el uso de generación pseudoaleatoria para piezas; requieren interpretar el contexto del juego antes de equipararlas a un riesgo explotable. La cobertura global, aunque disponible, se mantiene fuera de la valoración formal de Etapa 1 conforme a la metodología del capítulo.

Dimensión C: Métricas de Especificación y Documentación

Se contabilizaron los documentos de especificación nativos de cada herramienta y sus líneas físicas; se excluyeron *README*, dependencias, salidas de pruebas y evidencia dentro de *docs/*. Para concordar con la tabla de Calculadora, el ratio código/especificación divide las LOC *ncloc* de SonarQube entre las líneas físicas de especificación. También se indica el número de requisitos atómicos identificados en cada conjunto de documentos. El número de líneas mide volumen, no calidad semántica ni completitud por sí solo.

Tabla 9.18: Artefactos de especificación – Inicialización (Tetris Web)

Herramienta	Archivos	Líneas	Densidad (lín./arch.)	Código/spec	Requisitos
OpenSpec	4	442	110,50	2,03	30
Spec Kit	8	1.002	125,25	0,75	38
Tessl	2	136	68,00	5,79	34
BMAD	6	1.120	186,67	0,60	30
Kiro	3	608	202,67	0,82	35

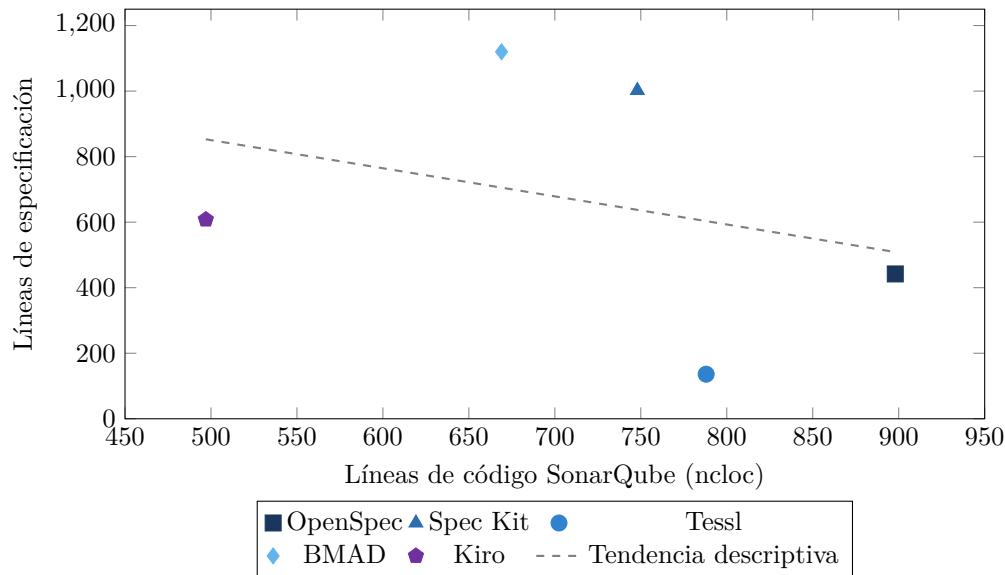


Figura 9.11: Relación entre LOC y líneas de especificación para las cinco variantes de Tetris. La línea punteada es el ajuste lineal de mínimos cuadrados de estos cinco puntos; no permite inferir causalidad ni predecir otras etapas.

BMAD es el caso más documentado: seis archivos principales y 1.120 líneas, con 30 requisitos funcionales trazables. El conteo excluye dos memorias de proceso y el estado YAML del sprint. Spec Kit conserva ocho archivos y 1.002 líneas; en ambas herramientas el ratio código/especificación es inferior a uno. Tssl concentra solo 136 líneas en dos archivos y alcanza el ratio mayor (5,79), pero aun así cubre 34 enunciados de requisito; su menor volumen no prueba automáticamente una mejor abstracción. Kiro concentra 608 líneas en tres archivos y la mayor densidad por archivo (202,67). El ajuste descriptivo de la Figura 9.11 desciende en esta muestra; con cinco observaciones y flujos documentales distintos, no permite inferir una relación causal. Las correcciones registradas muestran que contar requisitos no garantiza por sí solo fidelidad al alcance.

Dimensión D: Validación Funcional (Provisional)

El protocolo definitivo de aceptación funcional aún está en revisión. Como evidencia preliminar se presentan las suites automatizadas ejecutadas al cerrar la Etapa 1; las cinco compilaciones del cliente también pasaron. Spec Kit añadió cinco pruebas de *frontend* y Tssl dos; las otras tres variantes no dejaron una suite de *frontend* equivalente. Cada suite tiene distinto tamaño y cobertura de escenarios, de modo que el 100 % de pruebas exitosas dentro de ella no debe llamarse *Functional Acceptance*

Rate comparable. La tasa de aceptación, los defectos funcionales y los defectos críticos se mantienen como no determinados hasta ejecutar un mismo catálogo de casos contra los cinco productos.

Tabla 9.19: Evidencia preliminar de validación – Inicialización (Tetris Web)

Herramienta	Casos ejecutados	Exitosos	Fallidos	FAR común	Defectos/Críticos
OpenSpec	13	13	0	N/D	N/D
Spec Kit	23	23	0	N/D	N/D
Tessl	11	11	0	N/D	N/D
BMAD	13	13	0	N/D	N/D
Kiro	39	39	0	N/D	N/D

Nota: Casos = pruebas automatizadas propias de cada variante (*backend* y, cuando existen, *frontend*). FAR = *Functional Acceptance Rate*. N/D significa que no existe aún una medición común auditada, no que se hayan encontrado cero defectos.

Dimensión E: Intervención Humana (Provisional)

Los registros de invocaciones permiten contar solicitudes posteriores a la semilla inicial, pero no clasifican de manera uniforme cuáles fueron correcciones, pasos ordinarios del método o reintentos por fallas del servicio. Por ello la Tabla 9.20 presenta este indicador observable sin llamarlo número definitivo de rondas de corrección. No se cronometró por separado el tiempo de revisión humana ni se administró un protocolo común para registrar defectos hallados exclusivamente por personas; ambos resultados son N/D. Entre las intervenciones documentadas constan la reparación del escenario imposible de Spec Kit, la reducción del control por teclado de Tessl a las tres flechas requeridas y la eliminación de metas cuantitativas no solicitadas en PRD, épicas y especificación de BMAD. Estos ejemplos ilustran el papel de la revisión de especificaciones, pero no constituyen un recuento completo de defectos.

Tabla 9.20: Trazas de intervención observables – Inicialización (Tetris Web)

Herramienta	Solicitudes posteriores a la inicial	Tiempo de revisión humana	Defectos detectados solo por humanos
OpenSpec	5	N/D	N/D
Spec Kit	5	N/D	N/D
Tessl	7	N/D	N/D
BMAD	8	N/D	N/D
Kiro	0	N/D	N/D

9.2.3 Caso de Estudio 1.3: E-commerce

La inicialización del e-commerce se ejecutó de forma independiente con OpenSpec, Tessl, BMAD Method y Kiro a partir de una misma semilla canónica (SHA-256 con prefijo 45DDFDED6EBF). El alcance común comprendió un catálogo de productos, registro e inicio de sesión, carrito de compras, checkout simulado y persistencia relacional, con FastAPI en el *backend* y React con TypeScript en el *frontend*. Las métricas corresponden a la primera implementación congelada de cada herramienta, antes de aplicar correcciones posteriores. Las ejecuciones utilizaron OpenSpec 1.13.2, Tessl CLI 0.113.0, BMAD Method 6.12.1 y Kiro 1.1.14 en modo *Auto*; el análisis estático comparable se realizó con SonarQube Community Build 25.12.0.117093.